

Supplementary Materials for ‘Leaking Circuit Secrets: Gradient Leakage Attacks on Graph Neural Networks’

Appendix

I. BACKGROUND AND MOTIVATION

A. Gradient Leakage Attacks (GLAs)

GLAs [1], [2], [3] refer to a class of malicious techniques where attackers exploit gradient information, collected during the training or inference process of an ML model, to reconstruct sensitive details of the underlying input data. GLAs have been studied extensively in domains like image recognition, e.g., showing that visually similar images can be recovered from leaked gradients [1]. More recently, gradient leakage has also been explored for graph neural networks in generic learning settings. For example, [4] study gradient inversion attacks on GNNs for node- and graph-classification tasks, while related works investigate reconstruction of graph structures and node features from gradients in federated learning settings. In contrast to these efforts, our work focuses on circuit-trained GNNs and hardware-security applications, including gate classification and hardware Trojan detection, and studies the interplay between GNN architectures, circuit-specific features, and defense mechanisms. Table I summarizes prior art and ours.

B. GLAs on GNNs for Circuit Design – An Overlooked Threat

For circuit design, such attacks have been overlooked so far. To understand the motivation, an adversary might utilize GLAs on GNNs trained for circuit design and analysis to:

Reconstruct Sensitive Inputs. Recover design details embedded in the training set [1], [5]. For example, attackers may infer structural properties of security features like supervisor-mode logic, which may assist the analysis of such features in downstream attack scenarios.

Perform Membership Inference. Determine whether specific designs were used during training [6]. This may provide information that could assist the analysis of logic locking schemes [7] by identifying whether particularly vulnerable circuits or locking schemes were used for training [8].

Execute Model Inversion. Extract representative features or typical instances [9]. For example, attackers could identify patterns associated with false negatives for HT detection [10], which may inform the design of evasive structures to evade detection [11]. For another example, by identifying true positives for security assessment on logic locking, adversaries can learn which gate structures are most promising to attack.

Extract Model Parameters. Infer model parameters and/or reverse-engineering of proprietary ML algorithms [12], [13]. Attackers can reconstruct the learned feature space of GNNs

used for security assessment, gaining adversarial insights for, e.g., bypassing detection of counterfeiting or tampering [14].

These motivational examples underscore the need for robust assessment of GLAs on GNNs applied to sensitive circuits.

C. Threat Model

For an attacker to exploit gradient leakage, they must gain access to the gradients computed during training or inference. In this work, we consider a standard gradient leakage setting commonly adopted in prior literature [1], [2], where the attacker has access to model parameters and corresponding gradients (e.g., for a given training step or target node), and aims to reconstruct the underlying input features. This setting reflects a white-box or honest-but-curious scenario and serves as a canonical baseline to assess the inherent leakage of a model. This can be achieved through several means:

- **Model Extraction:** Attackers may gain access to pre-trained model weights post-deployment [19], allowing them to compute gradients on select inputs.
- **Side-Channel Attacks:** Adversaries can monitor hardware-level computations during training, like memory-access patterns, power consumption, or timing variations, to infer gradient updates without direct model access [20].
- **Federated Learning Scenarios:** If the model operates in a distributed training environment, gradients exchanged between clients and servers may be intercepted [13].
- **Adversarial Querying:** Attackers interacting with a deployed model can carefully craft queries to infer responses that, when analyzed systematically [21], expose gradients.

Unlike direct data theft, these means and subsequent GLAs can expose sensitive features in a stealthy manner, making them difficult to detect and reinforcing the need for robust defenses.

D. Mitigation Techniques

As explained, GLAs exploit the information contained in gradient updates to reconstruct sensitive input data. To mitigate such vulnerabilities, researchers have proposed several strategies that aim to obscure or limit the information that gradients reveal. However, as we later show, doing so is challenging for circuit-design tasks.

Differential Privacy. By injecting carefully calibrated noise into gradient updates, the training procedure ensures that the contribution of any individual data point is masked [21]. This

TABLE I: Comparison of prior art and ours.

Ref.	Models	Datasets	Metric rel _{l2}	Scope
[1]	CNNs (LeNet, AlexNet)	MNIST, CIFAR-10	≈ 0.01 (MNIST)	Recovers training data by inverting gradients from CNNs.
[15]	DNNs w/ GAN-based attack in collab. learning	MNIST, CIFAR-10	NA (visual reconstructions)	Exploits shared gradients to reconstruct sensitive inputs.
[16]	CNNs in federat. learning	CIFAR-10, ImageNet subsets	≈ 0.03 (CIFAR-10), ≈ 0.05 (ImageNet)	Attacks federated setups via gradient inversion.
[17]	DNNs in collab. learning	CIFAR-10, IMDB	NA (attribute inferences)	Shows unintended leakage through gradients.
[18]	Various CNNs	MNIST, CIFAR-10, ImageNet	Summarizes prior works (≈ 0.01 – 0.05)	Taxonomy of gradient inversion attacks, defenses.
[4]	GNNs (GCN, GraphSAGE)	Cora, Citeseer, PubMed	≈ 0.1 – 0.5 (node features)	Gradient inversion attacks on GNNs for node- and graph-level tasks in generic graph datasets.
Ours	GNNs (GCN, GraphSAGE, GIN, GAT)	ISCAS'85, EPFL, TrustHub	≈ 0.8 – 1.0 (gate classification), ≈ 1.2 – 2.2 (HT detection)	First comprehensive study of GLAs on circuit-trained GNNs for hardware-security tasks (gate classification, HT detection).

technique helps to prevent precise reconstruction of the original input while preserving general model performance¹

Gradient Clipping and Perturbation. Clipping restricts the magnitude of gradients, preventing large-scale updates that may leak more information [13]. Along with perturbations, this approach further obfuscates the training process.

Secure Aggregation Protocols. In distributed/federated learning settings, secure aggregation allows the server to receive only aggregated gradients from multiple participants [23]. By ensuring that individual gradient updates are never available in isolation, these protocols make it significantly more challenging for an adversary to reconstruct any specific data sample.

Model Compression and Quantization. Such techniques reduce the precision of gradient updates [24]. Lowering the resolution of the gradient information can inherently limit the granularity of data that is leaked, though this may come at cost for model accuracy.

Adversarial Training. Integrating adversarial examples during training not only makes models more robust to input perturbations but also helps to obfuscate gradient signals [13]. This approach deliberately alters learning dynamics so that extracted gradients become less indicative of original input features.

II. METHODOLOGY

We devise an end-to-end methodology for security evaluation of GNNs used for circuit design and analysis against GLAs. The workflow is outlined in Fig. 1 and explained next.

A Input Processing. The workflow begins with processing the circuit netlists, along with parameters for training etc.

B Graph Construction and Feature Setup for Sensitive Applications. Each netlist is converted into a graph where nodes represent gates and edges represent wires. While doing so, relevant features for the sensitive circuit design/analysis task of interest are employed. As motivated in Section I-B, adversarial techniques like membership inference or model inversion can be employed against GNNs devised for security assessment of, e.g., logic locking and HT detection. Accordingly, we consider classification of gate types as a representative proxy task relevant to logic locking analysis, and binary classification of malicious vs. benign gates as a representative task for guiding adversarial efforts to bypass HT detection.

Following [8], [25] and the wider literature, we use the following node features: *fan_in*, *fan_out*, *dist_to_output*, *is_primary_input*, *is_primary_output*, *is_internal*, *degree centrality*, *betweenness centrality*, *closeness centrality*,

clustering coefficient, *avg_fan_in_neighbors*, and *avg_fan_out_neighbors*. The gate type and HT assignment, respectively, are encoded as numerical and binary labels.

C GNN Training with GLA Mitigations. Training is performed using standard optimization techniques like gradient descent; a detailed description and implementation can be found in our release. At this stage, we also employ the GLA mitigation techniques outlined in Section I-D.

D Gradient Extraction. Here, we mimic attackers that extract gradients with respect to the model's parameters based on the loss observed at a targeted node during training.

E Input Reconstruction and Evaluation. Next, the leaked gradients are used for input reconstruction. That is, attackers iteratively update the dummy feature vector $\tilde{\mathbf{x}}_i$ until its corresponding gradients closely match the leaked gradients $\{G^{(l)}\}$. The attackers' objective can also be defined as:

$$\min_{\tilde{\mathbf{x}}_i} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{\mathbf{x}}_i, y) - G^{(l)} \right\|_2^2,$$

where $\mathcal{L}$ is the loss function and $\nabla_{W^{(l)}} \mathcal{L}(\tilde{\mathbf{x}}_i, y)$ denotes the gradients computed using the dummy feature.

The iterative procedure for updating $\tilde{\mathbf{x}}_i$ by attackers is below:

- 1) Initialization: Set $\tilde{\mathbf{x}}_i$ to a random vector. This is analogous to an initial guess or random noise.
- 2) Comparison: At each iteration, compute the gradients for $\tilde{\mathbf{x}}_i$ and measure the delta to the extracted gradients $\{G^{(l)}\}$.
- 3) Optimization: Update $\tilde{\mathbf{x}}_i$, using gradient descent, to minimize the objective function. This step ensures that $\tilde{\mathbf{x}}_i$ converges to a value that reproduces the leaked gradients.

A. Experiment Setup

1) Implementation: All experiments were implemented in Python, via Jupyter notebooks. For GNN models, we utilize the Deep Graph Library (DGL) [26]. For a case study on MNIST [27], which serves to contextualize the impact of GLAs on traditional image classification and simpler models, we use TensorFlow to devise a fully connected neural network (FCNN). We implement the proposed methodology (Section II) in full, including mitigation techniques. For the latter, we follow the default setups reported in their papers and codes. GNN and GLA hyper-parameters are provided in Table II; further details and source codes are available at https://anonymous.4open.science/r/Gradient_Attack_GCNN-11A4/README.md.

III. DEFENSE MECHANISMS FOR GRADIENT LEAKAGE ATTACK

A. Differential Privacy

To mitigate the risk of sensitive information being recovered via gradient inversion, we integrate a differential-privacy

¹In this work, differential privacy is implemented via gradient clipping and noise injection following prior practice [22], where clipping bounds per-sample gradients and noise is added to obscure individual contributions.

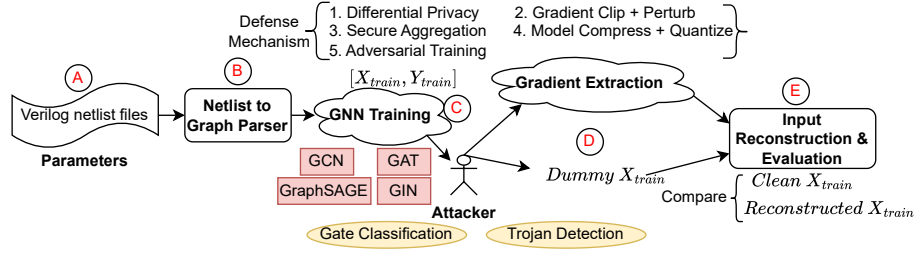


Fig. 1: Methodology for security evaluation of GNNs against GLAs.

TABLE II: Hyper-parameters for GNNs and GLAs.

Parameter	Gate Classification	HT Detection
Input Feature Dimension (d)	13	26
Hidden Layer Units	32	96
Number of Output Classes (C)	8	2
Learning Rate	0.01	0.01
Training Epochs	50	50
Loss Function	Cross-Entropy	Cross-Entropy
Optimizer	Adam	Adam
Attack Iterations	300	300
Dummy Optimizer Learning Rate	0.1	0.1
Regularization Weight (L2)	0.001	0.001

mechanism into the gradient extraction process, by perturbing the computed gradients with Gaussian noise. Let

$$G_{\text{true}}^{(l)} = \nabla_{W^{(l)}} \mathcal{L}(\hat{Y}, Y)$$

denote the true gradient of the loss function $\mathcal{L}$ with respect to the model parameters $W^{(l)}$ at layer l , where $\hat{Y}$ and Y are the model's prediction and the ground truth, respectively. We then make the gradient private by adding Gaussian noise:

$$G_{\text{dp}}^{(l)} = G_{\text{true}}^{(l)} + \mathcal{N}(0, \sigma^2 I),$$

where $\mathcal{N}(0, \sigma^2 I)$ denotes Gaussian noise with zero mean and covariance $\sigma^2 I$, and I is the identity matrix. In our experiments, we set $\sigma = 0.1$, i.e., the privacy noise multiplier is 0.1.

Consequently, the reconstruction objective for the gradient leakage attack is modified to minimize the difference between the noisy gradients $G_{\text{dp}}^{(l)}$ and the gradients computed from the dummy input $\tilde{X}$, given by:

$$\min_{\tilde{X}} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{X}) - G_{\text{dp}}^{(l)} \right\|_F^2.$$

In short, by injecting Gaussian noise with $\sigma = 0.1$ into the gradients, we seek to obscure sensitive information while maintaining sufficient utility for model training and evaluation.

B. Gradient Clipping and Perturbation

Here, we implement a defense mechanism that combines gradient clipping with perturbation. Let

$$G_{\text{true}}^{(l)} = \nabla_{W^{(l)}} \mathcal{L}(\hat{Y}, Y)$$

denote the true gradient of the loss $\mathcal{L}$ with respect to the model parameters $W^{(l)}$ at layer l . We first clip these gradients to restrict their magnitudes by enforcing

$$G_{\text{clip}}^{(l)} = \min \left(1, \frac{C}{\|G_{\text{true}}^{(l)}\|_2} \right) G_{\text{true}}^{(l)},$$

where the clipping threshold is set to $C = 1.0$. To further obscure the leaked information, we add Gaussian noise to produce the defended gradient:

$$G_{\text{def}}^{(l)} = G_{\text{clip}}^{(l)} + \mathcal{N}(0, \alpha^2 I),$$

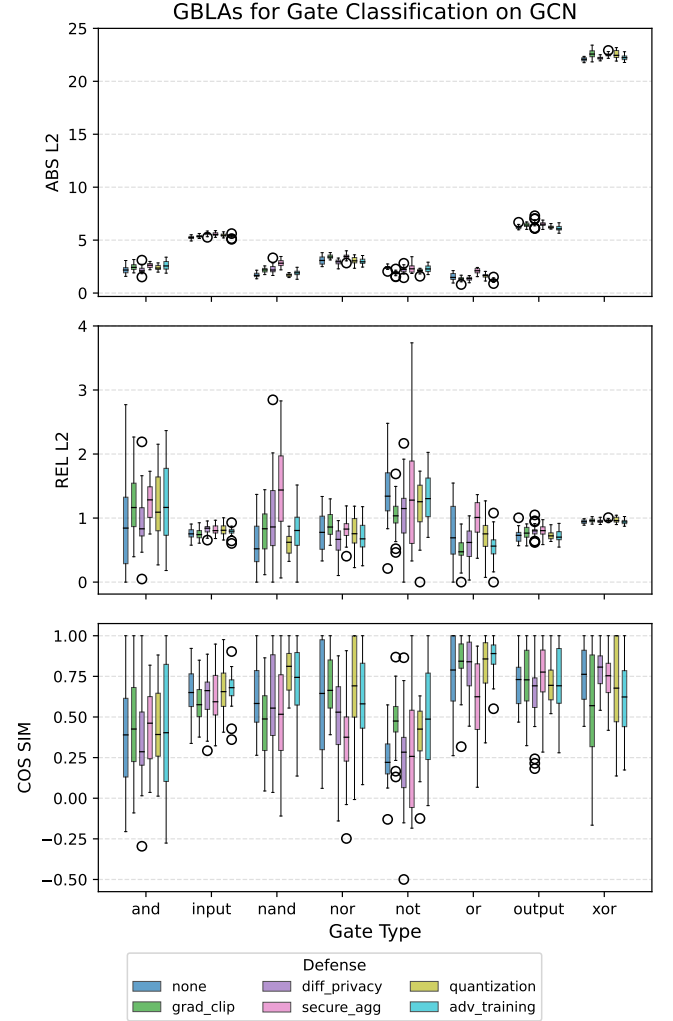


Fig. 2: Gate Classification: Reconstruction errors under different defense mechanism for GCN.

with $\alpha = 0.05$ and I being the identity matrix. The reconstruction objective then becomes

$$\min_{\tilde{X}} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{X}) - G_{\text{def}}^{(l)} \right\|_F^2,$$

where $\tilde{X}$ is the dummy input being optimized. This two-fold defense – clipping gradients to a maximum norm of 1.0 and perturbing them with a noise multiplier of 0.05 – effectively degrades the precision of the gradient signal available to the attacker, leading to higher reconstruction errors and improved privacy protection.

TABLE III: Comparison of GLA results across GNNs, tasks, and defenses. Aggregate GNN-wise averages and relative defense effects are summarized in Table II, while class-wise no-defense averages are summarized in Table III in the paper.

GNNs	Class Type	Gate Classification														
		No Defense			Diff. Priv. [21]			Grad. Clip. [13]			Secure Aggr. [23]			Compr. & Quant. [24]		
		abs_l2	rel_l2	cos_sim	abs_l2	rel_l2	cos_sim	abs_l2	rel_l2	cos_sim	abs_l2	rel_l2	cos_sim	abs_l2	rel_l2	cos_sim
GCN	AND	2.28	1.06	0.49	2.27	1.07	0.46	2.41	1.14	0.41	2.50	1.12	0.33	2.39	1.17	0.43
	INPUT	5.26	0.77	0.68	5.58	0.81	0.61	5.43	0.79	0.65	5.55	0.81	0.60	5.43	0.79	0.63
	NAND	1.81	0.77	0.73	2.22	0.94	0.60	2.30	1.01	0.60	2.70	1.18	0.40	1.70	0.60	0.79
	NOR	3.04	0.74	0.60	3.05	0.74	0.62	3.35	0.81	0.61	3.54	0.89	0.46	3.01	0.73	0.66
	NOT	2.43	1.46	0.26	2.29	1.25	0.34	1.89	1.06	0.49	2.32	1.34	0.29	2.10	1.18	0.39
	OR	1.45	0.65	0.76	1.34	0.57	0.81	1.36	0.50	0.87	2.03	0.92	0.55	1.57	0.64	0.77
	OUTPUT	6.27	0.75	0.76	6.54	0.78	0.63	6.42	0.76	0.73	6.41	0.76	0.69	6.27	0.75	0.74
	XOR	22.06	0.94	0.74	22.16	0.94	0.77	22.62	0.96	0.62	22.43	0.96	0.68	22.39	0.95	0.61
Graph SAGE	AND	2.58	1.24	0.30	2.48	1.12	0.37	2.57	1.27	0.34	3.28	1.70	0.14	2.72	1.27	0.34
	INPUT	6.02	0.88	0.56	5.86	0.86	0.60	6.23	0.91	0.44	6.06	0.88	0.54	6.24	0.91	0.45
	NAND	2.01	0.86	0.66	2.36	1.02	0.53	2.37	1.04	0.64	2.84	1.24	0.42	1.70	0.71	0.75
	NOR	3.43	0.84	0.50	3.32	0.83	0.52	3.56	0.88	0.51	3.71	0.93	0.44	3.41	0.89	0.51
	NOT	2.74	1.51	0.21	2.71	1.57	0.29	2.62	1.51	0.12	2.28	1.30	0.26	2.99	1.79	0.08
	OR	1.01	0.41	0.90	1.17	0.49	0.82	1.29	0.53	0.86	1.70	0.75	0.65	1.27	0.51	0.82
	OUTPUT	7.09	0.84	0.68	7.38	0.88	0.53	7.32	0.87	0.59	7.36	0.88	0.55	7.24	0.86	0.58
	XOR	21.93	0.93	0.63	22.41	0.95	0.51	22.21	0.95	0.54	22.32	0.95	0.54	22.02	0.94	0.62
GIN	AND	3.50	1.77	0.10	3.72	1.97	0.07	3.68	1.92	0.13	3.43	1.83	0.18	2.93	1.39	0.23
	INPUT	6.87	1.00	0.24	6.89	1.01	0.24	6.51	0.95	0.36	6.64	0.97	0.29	7.00	1.02	0.20
	NAND	3.71	1.74	0.29	3.91	2.04	0.16	4.29	2.12	0.09	4.11	2.02	0.09	4.18	2.03	0.06
	NOR	4.85	1.29	0.09	4.52	1.20	0.26	5.15	1.36	0.03	5.11	1.36	0.10	4.55	1.23	0.25
	NOT	2.91	1.64	0.23	3.28	1.90	0.07	3.47	2.08	0.13	3.27	1.99	0.20	3.54	2.14	0.14
	OR	3.67	1.86	0.10	3.48	1.83	0.17	4.19	2.11	-0.07	3.67	1.85	0.21	3.73	1.95	0.05
	OUTPUT	8.17	0.97	0.22	7.69	0.91	0.40	8.09	0.96	0.30	8.41	1.00	0.18	8.25	0.98	0.23
	XOR	22.81	0.97	0.27	22.72	0.97	0.32	22.60	0.96	0.32	22.78	0.97	0.28	22.78	0.97	0.27
GAT	AND	1.93	0.90	0.58	2.21	1.06	0.51	2.16	1.02	0.45	2.36	1.08	0.38	2.08	1.02	0.53
	INPUT	5.56	0.81	0.71	5.52	0.81	0.70	5.64	0.82	0.65	5.66	0.83	0.65	5.73	0.84	0.65
	NAND	1.88	0.66	0.79	2.04	0.79	0.71	1.79	0.67	0.78	2.45	0.94	0.48	1.78	0.65	0.84
	NOR	3.24	0.79	0.61	3.16	0.77	0.69	3.05	0.73	0.68	3.54	0.92	0.53	3.16	0.76	0.64
	NOT	1.70	0.91	0.48	1.89	1.05	0.40	2.02	1.12	0.40	1.87	1.03	0.38	2.13	1.25	0.40
	OR	1.53	0.64	0.81	1.59	0.68	0.78	1.75	0.71	0.73	2.09	0.91	0.57	1.42	0.60	0.90
	OUTPUT	6.36	0.76	0.86	6.41	0.76	0.83	6.38	0.76	0.85	6.34	0.75	0.85	6.34	0.75	0.85
	XOR	20.84	0.89	0.88	21.07	0.90	0.87	20.73	0.88	0.89	20.96	0.89	0.88	20.69	0.88	0.89
HT Detection																
GCN	Clean	7.59	1.57	0.17	7.16	1.38	0.22	7.79	1.46	0.14	8.09	1.49	0.10	7.74	1.50	0.11
	HT	6.48	2.22	0.13	6.73	2.26	0.04	6.91	2.24	-0.04	6.49	2.12	0.05	6.59	2.08	0.00
Graph SAGE	Clean	3.34	0.85	0.50	4.69	1.41	0.22	3.34	1.02	0.38	5.57	1.30	0.19	4.24	1.34	0.34
	HT	4.95	0.49	0.89	1.02	0.40	0.94	0.99	0.41	0.91	8.63	2.57	0.17	1.88	0.69	0.87
GIN	Clean	4.66	1.20	-0.05	4.33	1.24	-0.02	3.93	1.19	-0.02	5.55	1.36	-0.10	3.99	1.12	0.02
	HT	8.07	1.55	0.15	4.69	1.41	0.17	3.56	1.32	0.26	5.42	1.37	0.18	4.78	1.42	0.19
GAT	Clean	3.8	0.9	-0.05	4.2	1.0	-0.03	3.5	0.85	-0.02	5.0	1.2	-0.08	4.0	0.95	0.01
	HT	6.5	1.4	0.18	5.0	1.3	0.17	5.8	1.35	0.22	7.2	1.45	0.16	5.5	1.35	0.19

C. Secure Aggregation Protocols

Here, we apply secure aggregation protocols, wherein gradients are computed and averaged over multiple nodes instead of individual samples. This aggregated approach limits the attacker's ability to reconstruct specific node features while preserving the integrity of model updates.

Let a group of N_{group} nodes be selected from a given class, each with its corresponding feature X_i . The aggregated feature for the group is computed as:

$$X_{\text{agg}} = \frac{1}{N_{\text{group}}} \sum_{i=1}^{N_{\text{group}}} X_i.$$

Similarly, the aggregated gradients for the group are computed using the mean loss over all selected nodes:

$$G_{\text{agg}}^{(l)} = \frac{1}{N_{\text{group}}} \sum_{i=1}^{N_{\text{group}}} \nabla_{W^{(l)}} \mathcal{L}(X_i, Y_i),$$

where $\mathcal{L}(X_i, Y_i)$ represents the loss function for node i .

In our implementation, we set $N_{\text{group}} = 5$ and simulate an attacker attempting to reconstruct X_{agg} rather than individual node features. The reconstruction objective now becomes:

$$\min_{\tilde{X}_{\text{agg}}} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{X}_{\text{agg}}) - G_{\text{agg}}^{(l)} \right\|_F^2.$$

Since the attacker is only provided with the averaged gradient and feature information, individual variations within the group are masked, notably degrading inversion performance.

D. Model Compression and Quantization

Here, we use model compression and quantization techniques to restrict the precision of model parameters, obfuscating critical information that could be exploited by an attacker. This approach involves two key defenses: (1) quantization, where model outputs are discretized to lower precision, and (2) compression through weight pruning, where small-valued parameters are removed to minimize information leakage.

1) *Quantization*: During the forward pass of the GCNN, numerical outputs are quantized using:

$$X_{\text{quant}} = \frac{\text{round}(X \cdot Q)}{Q},$$

where $Q = 255$ simulates 8-bit fixed-point representation. This transformation limits the granularity of feature updates, reducing inversion fidelity.

2) *Compression via Pruning*: After backpropagation, small-magnitude model weights are set to zero:

$$W_{\text{compressed}}^{(l)} = \begin{cases} 0, & \text{if } |W^{(l)}| < \tau \\ W^{(l)}, & \text{otherwise} \end{cases}$$

where $\tau = 0.01$ is the pruning threshold. This process eliminates weak connections in the NN, making gradient inversion less effective.

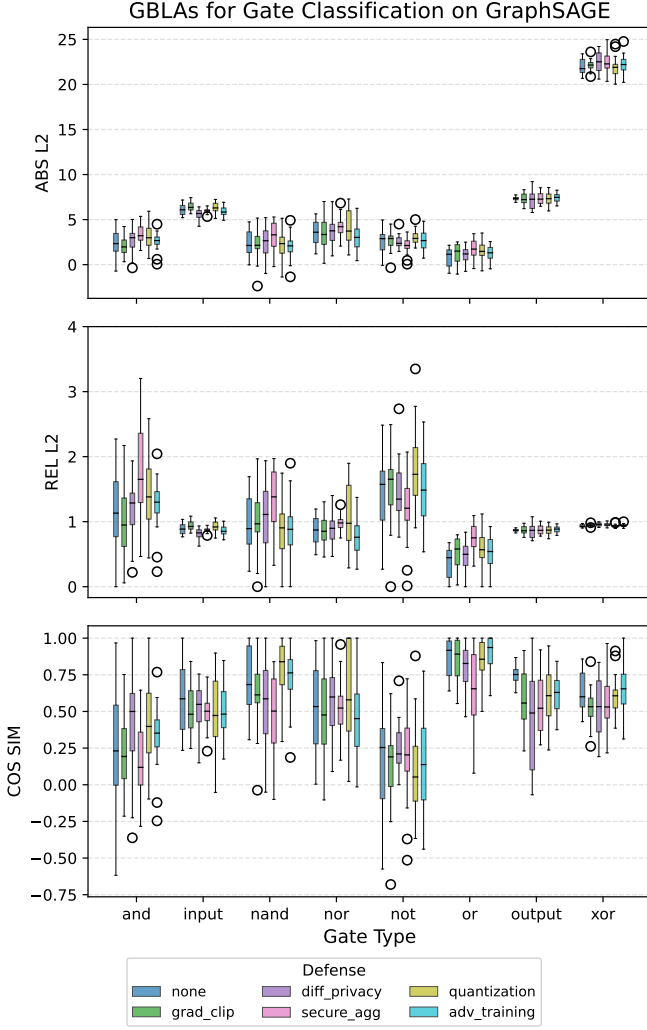


Fig. 3: Gate Classification: Reconstruction errors under different defense mechanism for GraphSAGE.

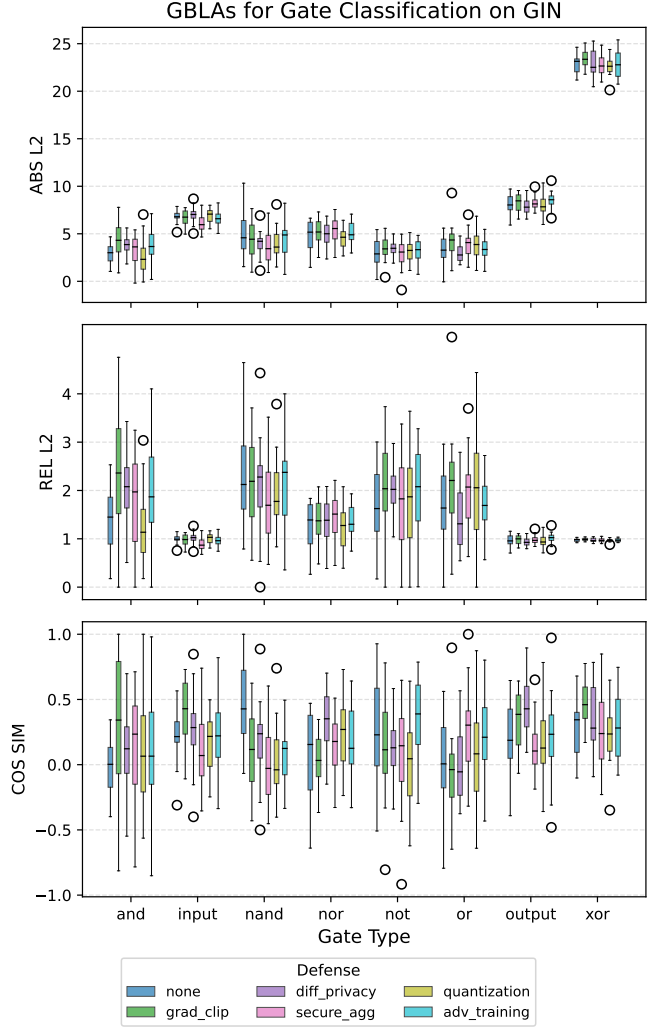


Fig. 4: Gate Classification: Reconstruction errors under different defense mechanism for GIN.

E. Adversarial Training

Here, we introduce adversarial perturbations into the model’s training process to improve its resilience against GBLAs. By exposing the model to carefully crafted perturbations during training, it learns representations that are less susceptible to inversion attacks.

1) *Generating Adversarial Perturbations:* To generate adversarial examples, we use the Fast Gradient Sign Method (FGSM), where the perturbation applied to the feature matrix X is computed as:

$$X_{\text{adv}} = X + \epsilon \cdot \text{sign}(\nabla_X \mathcal{L}(X, Y)),$$

where ϵ is the perturbation strength (set to 0.1 in our experiments), $\mathcal{L}(X, Y)$ is the loss function, and $\nabla_X \mathcal{L}$ represents the gradient of the loss with respect to the input features.

2) *Adversarial Training Procedure:* During training, instead of using the original features, we compute adversarial perturbations and feed the model with X_{adv} , reinforcing its ability to generalize across perturbed data:

$$\min_W \sum_{i=1}^N \mathcal{L}(f_W(X_{\text{adv},i}), Y_i).$$

This forces the model to optimize for robustness rather than merely fitting the clean training data.

IV. PLOTS FOR DIFFERENT DEFENSES UNDER GNNs

The box plots for different metrics (`abs_l2`, `rel_l2`, and `cos_sim`) for both the gate classification and the hardware Trojan detection on GCN, GraphSAGE, GIN, and GAT models are shown in Figs. 2, 3, 4, 5, 6, 7, and 8 respectively.

REFERENCES

- [1] L. Zhu, Z. Liu, and S. Han, “Deep leakage from gradients,” *Advances in neural information processing systems*, vol. 32, 2019.
- [2] H. Gong, L. Jiang, X. Liu, Y. Wang, O. Gastro, L. Wang, K. Zhang, and Z. Guo, “Gradient leakage attacks in federated learning,” *Artificial Intelligence Review*, vol. 56, no. Suppl 1, pp. 1337–1374, 2023.
- [3] D. Scheliga, P. Mäder, and M. Seeland, “Dropout is not all you need to prevent gradient leakage,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 8, 2023, pp. 9733–9741.
- [4] D. A. Sinha, R. Du, Y. Liu, A. Markopolou, and Y. Shen, “Gradient inversion attack on graph neural networks,” *arXiv preprint arXiv:2411.19440*, 2024.
- [5] H. Fang, Y. Qiu, H. Yu, W. Yu, J. Kong, B. Chong, B. Chen, X. Wang, S.-T. Xia, and K. Xu, “Privacy leakage on dnnns: A survey of model inversion attacks and defenses,” *arXiv preprint arXiv:2402.04013*, 2024.

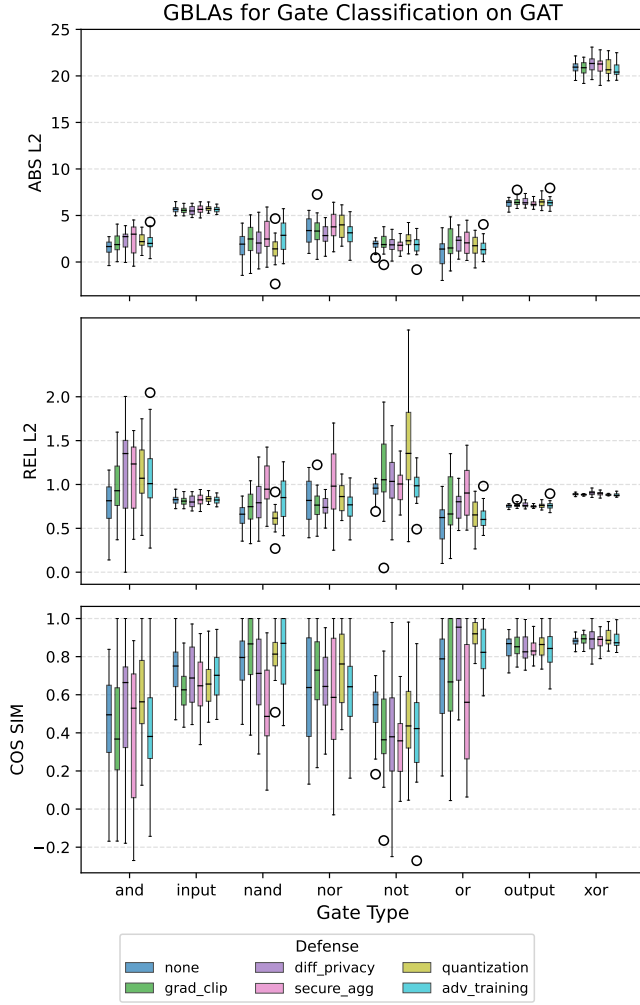


Fig. 5: Gate Classification: Reconstruction errors under different defense mechanism for GAT.

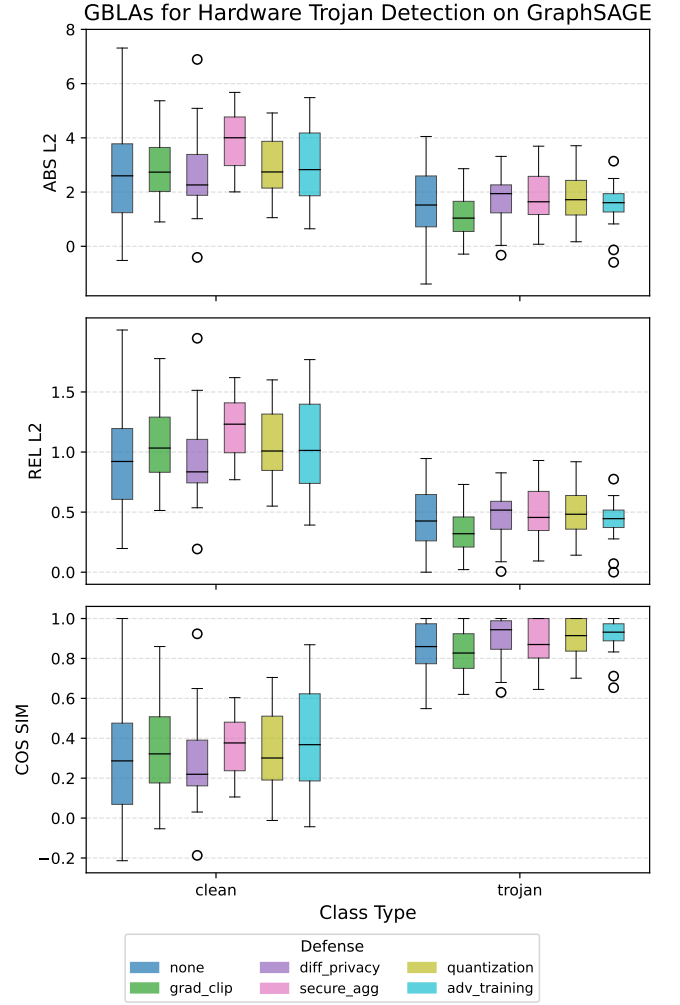


Fig. 6: Hardware Trojan Detection: Reconstruction errors under different defense mechanism for GraphSAGE.

- [6] H. Hu, Z. Salcic, L. Sun, G. Dobbie, P. S. Yu, and X. Zhang, "Membership inference attacks on machine learning: A survey," *ACM Computing Surveys (CSUR)*, vol. 54, no. 11s, pp. 1–37, 2022.
- [7] H. M. Kamali, K. Z. Azar, F. Farahmandi, and M. Tehranipoor, "Advances in logic locking: Past, present, and prospects," *Cryptology ePrint Archive*, 2022.
- [8] L. Alrahis, S. Patnaik, M. Shafique, and O. Sinanoglu, "Omla: An oracle-less machine learning-based attack on logic locking," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 3, pp. 1602–1606, 2021.
- [9] J. Song and D. Namiot, "A survey of the implementations of model inversion attacks," in *International Conference on Distributed Computer and Communication Networks*. Springer, 2022, pp. 3–16.
- [10] K. Kunal, T. Dhar, M. Madhusudan, J. Poojary, A. K. Sharma, W. Xu, S. M. Burns, J. Hu, R. Harjani, and S. S. Sapatnekar, "Gnn-based hierarchical annotation for analog circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 9, pp. 2801–2814, 2023.
- [11] A. Bhattacharyay, S. Yang, J. Cruz, P. Chakraborty, S. Bhunia, and T. Hoque, "An automated framework for board-level trojan benchmarking," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 2, pp. 397–410, 2022.
- [12] R. Torrance and D. James, "The state-of-the-art in ic reverse engineering," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2009, pp. 363–381.
- [13] W. Wei and L. Liu, "Gradient leakage attack resilient deep learning," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 303–316, 2021.
- [14] T. Meade, S. Zhang, and Y. Jin, "Netlist reverse engineering for high-level functionality reconstruction," in *2016 21st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2016, pp. 655–660.
- [15] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the gan: information leakage from collaborative deep learning," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 603–618.
- [16] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients-how easy is it to break privacy in federated learning?" *Advances in neural information processing systems*, vol. 33, pp. 16937–16947, 2020.
- [17] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *2019 IEEE symposium on security and privacy (SP)*. IEEE, 2019, pp. 691–706.
- [18] R. Zhang, S. Guo, J. Wang, X. Xie, and D. Tao, "A survey on gradient inversion: Attacks, defenses and future directions," *arXiv preprint arXiv:2206.07284*, 2022.
- [19] H. Wen, Y. Li, Z. Zhang, S. Jiang, X. Ye, Y. Ouyang, Y. Zhang, and Y. Liu, "Adaptivenet: Post-deployment neural architecture adaptation for diverse edge environments," in *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, 2023, pp. 1–17.
- [20] H. Chabanne, J.-L. Danger, L. Guiga, and U. Kühne, "Side channel attacks for architecture extraction of neural networks," *CAAI Transactions on Intelligence Technology*, vol. 6, no. 1, pp. 3–16, 2021.
- [21] Y. Li, S. Cheng, H. Su, and J. Zhu, "Defense against adversarial attacks via controlling gradient leaking on embedded manifolds," in *European Conference on Computer Vision*. Springer, 2020, pp. 753–769.
- [22] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 308–318.

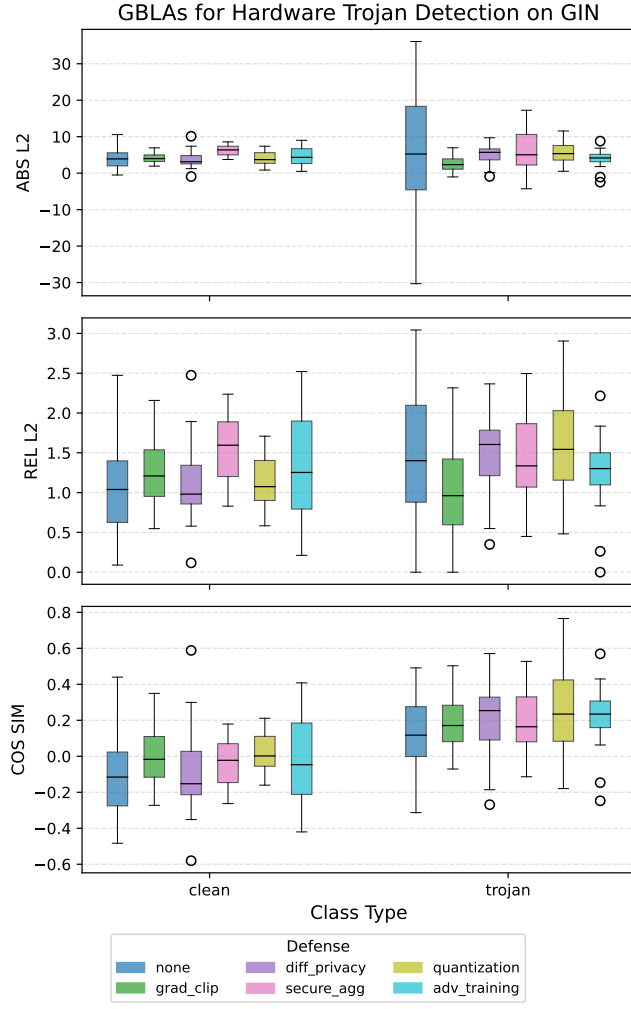


Fig. 7: Hardware Trojan Detection: Reconstruction errors under different defense mechanism for GIN.

- [23] Z. Wang, Z. Chang, J. Hu, X. Pang, J. Du, Y. Chen, and K. Ren, "Breaking secure aggregation: Label leakage from aggregated gradients in federated learning," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 151–160.
- [24] Q. Liu and W. Wen, "Model compression hardens deep neural networks: A new perspective to prevent adversarial attacks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 1, pp. 3–14, 2021.
- [25] H. Lashen, L. Alrahis, J. Knechtel, and O. Sinanoglu, "Trojansaint: Gate-level netlist sampling-based inductive learning for hardware trojan detection," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023.
- [26] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai *et al.*, "Deep graph library: A graph-centric, highly-performant package for graph neural networks," *arXiv preprint arXiv:1909.01315*, 2019.
- [27] A. Baldominos, Y. Saez, and P. Isasi, "A survey of handwritten character recognition with mnist and emnist," *Applied Sciences*, vol. 9, no. 15, p. 3169, 2019.

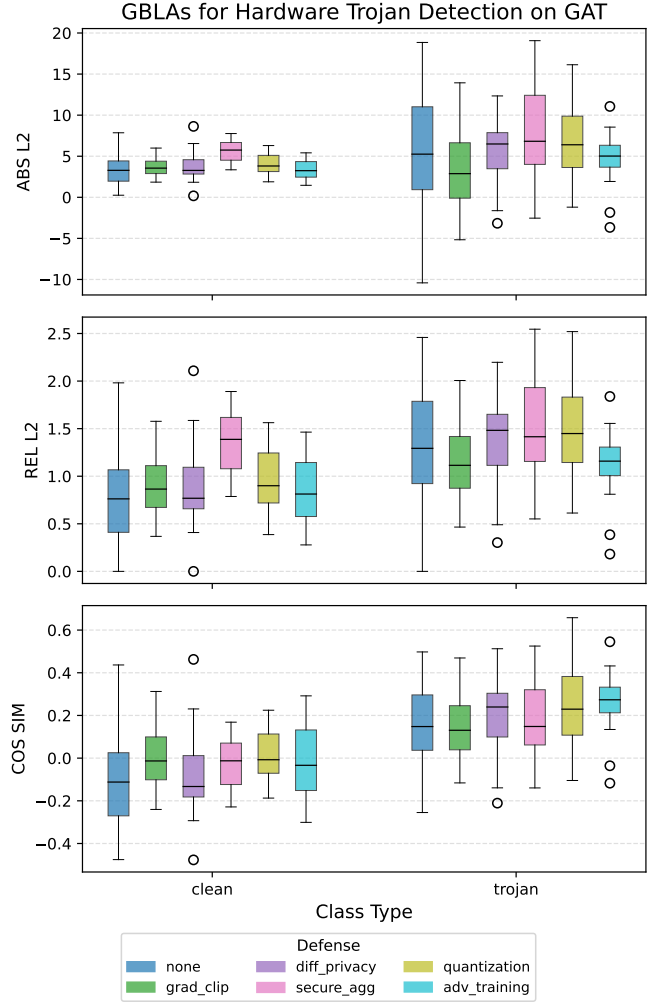


Fig. 8: Hardware Trojan Detection: Reconstruction errors under different defense mechanism for GAT.